# Technical Documentation

## 1. Introduction

**What is RAG?** Retrieval-Augmented Generation (RAG) is a framework that improves Large Language Model (LLM) responses by dynamically fetching facts from an external knowledge base to ground the generation process.

**Why it is needed:** Base LLMs hallucinate internal company policies and pricing because they aren't trained on proprietary data. RAG grounds the model in facts without the immense cost and rigidity of fine-tuning.

**Use Case Overview:** A Tier-1 customer support agent that answers routine queries instantly using company PDFs, but intelligently pauses and flags human agents for complex issues.

## 2. System Architecture & Components

The system diverges from simple LangChain linear chains by utilizing **LangGraph**. Nodes represent functional units (Embedding, Generation, Routing) while a shared `State` object acts as memory. This cyclic architecture allows for loops (e.g., self-correction if an answer is poor) and state interruptions (waiting for a human).

## 3. Design Decisions

- **Chunk Size Choice:** 1000 characters with 200 overlap. 1000 characters perfectly captures standard support paragraphs. The 200 overlap prevents critical sentences from being chopped in half at chunk boundaries.
- **Embedding Strategy:** We use local `all-MiniLM-L6-v2` models via HuggingFace. Local embeddings drastically speed up the DB ingestion process and cost $0, compared to OpenAI's API.

- **Prompt Design Logic:** Prompts are designed with explicit fallback instructions. "If the context does not contain the answer, do not guess. State that you need to transfer the user."

## 4. Workflow & Conditional Logic

**Intent Detection:** A lightweight LLM router classifies queries into categories. "Hello" bypasses the vector search entirely, saving time and tokens.
**Routing Decisions:** When generation is complete, an evaluator node acts as a judge. It scores the output. If confidence is high, the state transitions to the `END` node. If low, an edge routes it to the `HITL` node.

## 5. HITL Implementation Details

**Role:** The ultimate safety net against AI hallucinations.
**Benefits:** Protects brand reputation by ensuring sensitive issues are handled by humans. Crucially, the human receives the entire LangGraph `State`, meaning the customer never has to repeat themselves.

## 6. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed:** Setting Retriever K=5 provided better context but slowed down the LLM and risked prompt distraction. K=3 was the optimal balance.
- **Cost vs. Performance:** Migrating to larger 70B models increases reasoning capability drastically but introduces slight token-generation latency.

## 7. Testing Strategy & Future Enhancements

Testing utilizes a query matrix: In-Domain ("Reset password"), Out-of-Domain ("Who is the president?"), and Ambiguous ("It doesn't work").

**Future Enhancements:** Implementing a Postgres Checkpointer for long-term session memory, and adding multi-namespace Vector DB support for different product lines.